

TUP



Gestión de Desarrollo de Software

Unidad I

Pruebas Caja Blanca y Negra





Temario

Técnicas de Prueba

- Caja Blanca
- Caja Negra





Pruebas

Una estrategia para la prueba de software debe incluir pruebas de bajo nivel, que son necesarias para verificar que un pequeño segmento de código fuente se implementó correctamente, así como pruebas de alto nivel, que validan las principales funciones del sistema a partir de los requerimientos del cliente.

Inicialmente, las pruebas se enfocan en cada componente de manera individual, lo que garantiza que funcionan adecuadamente como unidad. De ahí el nombre de **prueba de unidad**. Esta prueba utiliza mucho de las técnicas de prueba que ejercitan rutas específicas en una estructura de control de componentes para asegurar una cobertura completa y la máxima detección de errores. A continuación, los componentes deben ensamblarse o integrarse para formar el paquete de software completo. La **prueba de integración** aborda los conflictos asociados con los problemas duales de verificación y construcción de programas. Un componente puede tener un inadvertido efecto adverso sobre otro; las subfunciones, cuando se combinan, pueden no producir la función principal deseada; la imprecisión aceptable individualmente puede magnificarse a niveles inaceptables, etc. Durante la integración, se usan más las técnicas de diseño de casos de prueba que se enfocan en entradas y salidas. La **prueba de validación** proporciona la garantía final de que el software cumple con todos los requerimientos informativos, funcionales, de comportamiento y de rendimiento. El software, una vez validado, debe combinarse con otros elementos del sistema (por ejemplo, hardware, personal, bases de datos). La **prueba del sistema** verifica que todos los elementos se mezclan de manera adecuada y que se logra el funcionamiento/rendimiento global del sistema.



Pruebas

La meta de testear es encontrar errores, en consecuencia, las pruebas deberán tener por objetivo encontrar la mayor cantidad de errores con el mínimo esfuerzo.

Una buena prueba debe tener como característica:

- **Alta probabilidad de encontrar un error.**
- **No ser redundante:** no realizar una prueba que tenga el mismo propósito que otra.
- **Ser “la mejor de la camada”:** en un grupo de pruebas que cuentan con objetivos similares, las limitaciones de tiempo y recursos pueden obligar la ejecución de sólo un subconjunto de dichas pruebas, en consecuencia, se debe recurrir a la prueba que cuente con la mayor probabilidad de descubrir toda una clase de errores.
- **No debe ser demasiado simple pero tampoco demasiado compleja.**





Prueba de Caja Blanca

La *prueba de caja blanca* del software se basa en el examen cercano de los detalles de procedimiento. Las rutas lógicas a través del software y las colaboraciones entre componentes se ponen a prueba al revisar conjuntos específicos de condiciones y/o bucles.

Utilizar los métodos de prueba de caja blanca permite generar casos de prueba que:

1. Garanticen que todas las rutas independientes dentro de un módulo se revisaron al menos una vez.
2. Revisen todas las decisiones lógicas en sus lados verdadero y falso.
3. Ejecuten todos los bucles en sus fronteras y dentro de sus fronteras operativas.
4. Revisen estructuras de datos internas para garantizar su validez.

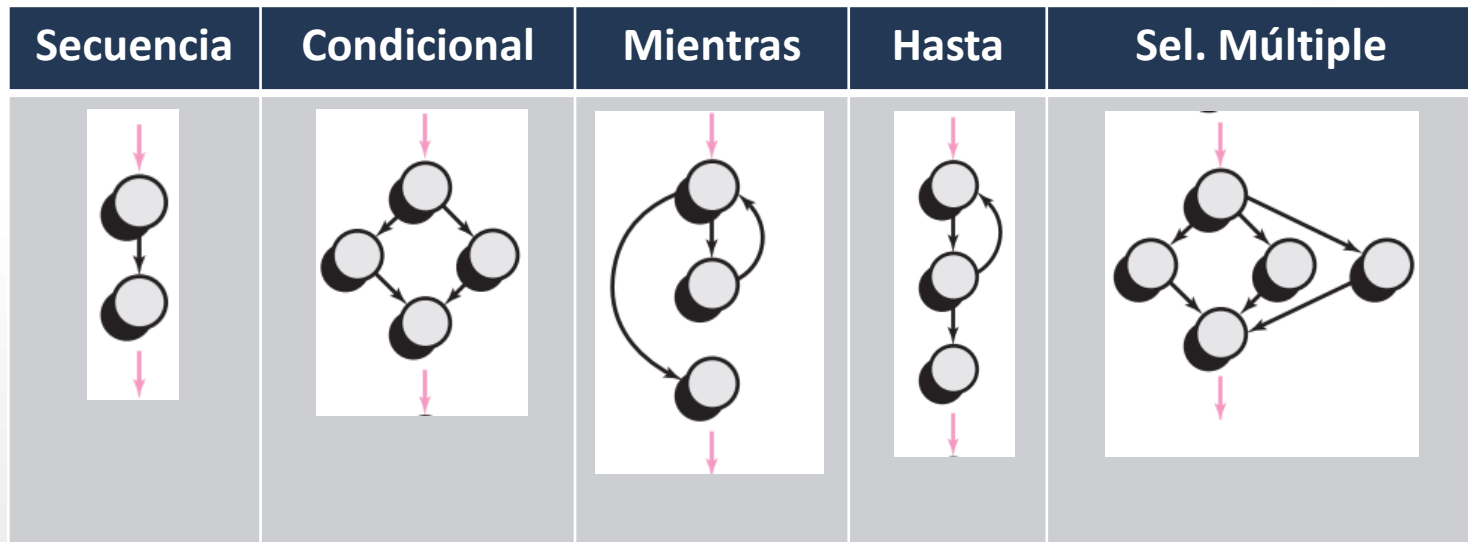




Prueba de Caja Blanca

La *prueba de ruta o trayectoria básica* es una técnica de prueba de caja blanca propuesta por primera vez por Tom McCabe. El método de ruta básica permite al diseñador de casos de prueba obtener una medida de complejidad lógica (complejidad ciclomática) del diseño de un módulo y usar esta medida como guía para definir un conjunto básico de rutas de ejecución. Los casos de prueba derivados del conjunto básico, garantizan la ejecución de todo enunciado, al menos una vez durante la prueba.

El primer paso es confeccionar el grafo asociado al procedimiento a testear. El grafo o gráfico de flujo muestra el flujo de control lógico y cada estructura de control se representa a través de un gráfico de flujo preestablecido.





Prueba de Caja Blanca

Cada circulo se denomina **nodo** y cada flecha se denomina **arista**.

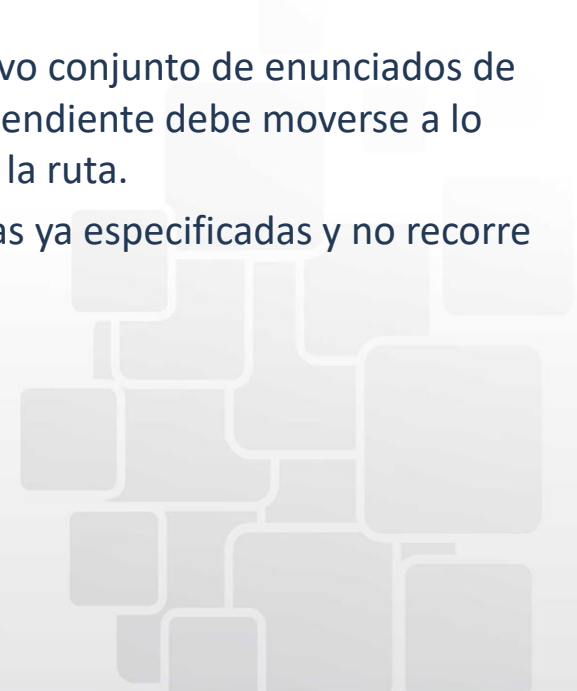
Las áreas acotadas por aristas y nodos se llaman **regiones**. El área afuera del gráfico también se considera una región.

Cuando en un procedimiento se encuentran condiciones compuestas, la generación de un gráfico de flujo se vuelve ligeramente más complicada. Una condición compuesta ocurre cuando uno o más operadores booleanos (OR, AND, NAND, NOR lógicos) se presentan en un enunciado condicional. Cuando se encuentra una condición compuesta, se crea un nodo separado para cada una de las condiciones.

Cada nodo que contiene una condición se llama **nodo predicado** y se caracteriza por tener dos o más aristas que emanan de él.

Una **ruta independiente** es cualquiera que introduce al menos un nuevo conjunto de enunciados de procesamiento o una nueva condición en el programa. Una ruta independiente debe moverse a lo largo de al menos una arista que no se haya recorrido antes de definir la ruta.

Una ruta no se considera independiente si es una combinación de rutas ya especificadas y no recorre alguna arista nueva.





Prueba de Caja Blanca

Se debe tener en cuenta que el conjunto básico no es único. De hecho, para un diseño de procedimiento dado, pueden derivarse algunos conjuntos básicos diferentes.

Para obtener un conjunto de rutas básicas pueden seguirse varios métodos, uno de ellos es conocido como “**simplificado**”, el cual comienza seleccionando el camino más corto de principio a fin y luego va buscando segmentos no recorridos hasta completar el número de rutas requeridas. Este método tiende a recorrer primero los caminos de excepción, los errores y salidas de emergencia.

¿Cómo saber cuántas rutas buscar?

El cálculo de la complejidad ciclomática proporciona la respuesta. La **complejidad ciclomática** es una medición de software que proporciona una evaluación cuantitativa de la complejidad lógica de un programa. El valor calculado por la complejidad ciclomática define el número de rutas independientes del conjunto básico de un programa.

En otras palabras, define el número de pruebas que se deben ejecutar a fin de asegurar que todos los enunciados se ejecutaron al menos una vez.

La complejidad ciclomática puede calcularse como:

1. $V(G) = \text{Cant. de Regiones}$
2. $V(G) = \text{Cant. Aristas} - \text{Cant. Nodos} + 2$
3. $V(G) = \text{Cant. Nodos Predicados} + 1$





Prueba de Caja Negra

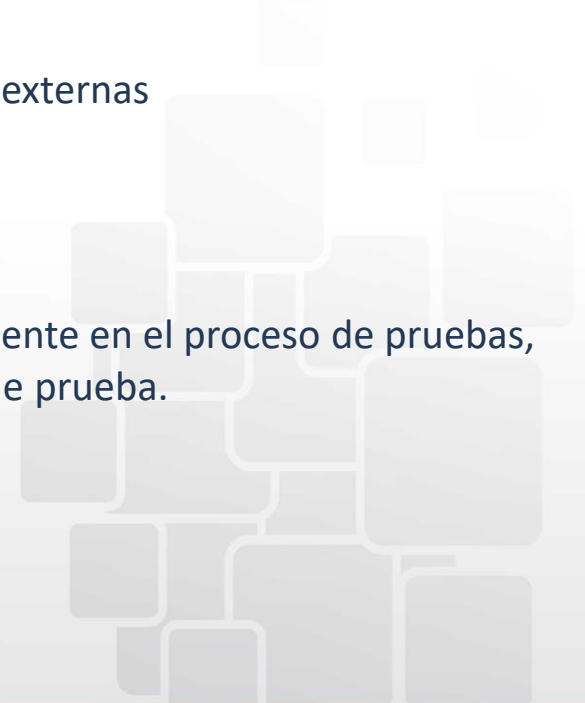
La *prueba de caja negra* se refiere a las pruebas que se llevan a cabo sobre la interfaz del software. Una prueba de caja negra examina algunos aspectos fundamentales de un sistema con poca preocupación por la estructura lógica interna del software. Se enfoca en los requerimientos funcionales del software.

Las pruebas de caja negra no son una alternativa para las técnicas de caja blanca. En vez de ello, es un enfoque complementario que es probable que descubra una clase de errores diferente que los métodos de caja blanca.

Las pruebas de caja negra intentan encontrar errores en las categorías siguientes:

1. Funciones incorrectas o faltantes
2. Errores de interfaz
3. Errores en las estructuras de datos o en el acceso a bases de datos externas
4. Errores de comportamiento o rendimiento
5. Errores de inicialización y terminación

A diferencia de las pruebas de caja blanca, que se realizan tempranamente en el proceso de pruebas, la prueba de caja negra tiende a aplicarse durante las últimas etapas de prueba.





Prueba de Caja Negra

La **partición de equivalencia** es un método de prueba de caja negra que divide el dominio de entrada de un programa en clases de datos de los que pueden derivarse casos de prueba.

El diseño de casos de prueba para la partición de equivalencia se basa en una evaluación de las **clases de equivalencia** para una condición de entrada.

Las clases de equivalencia pueden definirse de acuerdo con los siguientes lineamientos:

1. Si una condición de entrada especifica un rango, se define una clase de equivalencia válida y dos inválidas.
2. Si una condición de entrada requiere un valor específico, se define una clase de equivalencia válida y dos inválidas.
3. Si una condición de entrada especifica un miembro de un conjunto, se define una clase de equivalencia válida y una inválida.
4. Si una condición de entrada es booleana, se define una clase válida y una inválida



Muchas gracias por su atención.

